Universidad Nacional de San Agustín de Arequipa
Escuela Profesional de Ciencia de la Computación
Curso: Sistemas Operativos

GUÍA DE LABORATORIO 06

Comunicación entre procesos usando lenguaje C para Linux

Docentes: Prof. Rolando Jesús Cárdenas Talavera
Prof. Marcela Quispe Cruz

1 Competencia del Curso

El alumno comprenderá y analizará el funcionamiento interno de los sistemas operativos desde la utilización de comandos hasta la programación de procesos.

2 Competencia del Laboratorio

El alumno deberá de probar, analizar y explicar el comportamiento de los diferentes medios de comunicación que existen entre procesos para un intercambio de datos de forma segura.

3 Equipos y Materiales

- Un computador.
- Sistema Operativo Linux (Cualquier distribución es valida)
- Compilador GCC para Linux

4 Marco Teórico

Cualquier proceso en Linux tiene la posibilidad de establecer un medio de comunicación con otro procesos. Estos modelos de comunicación pueden ser por memoria compartida o por envió de mensajes. Un mecanismo de comunicación que el sistema Linux pone a disposición son los *pipes*.

Los *pipes*, generados mediante la función *pipe()*, abre una "tubería" (o canal) de comunicación y nos devuelve dos descriptores de fichero abiertos, uno por cada extremo de la tubería. Por un extremo de ellos se puede leer datos (con la función *read()*) que han sido escritos por el segundo proceso (con el uso de la función *write()*).

La ventaja de este mecanismo, es que si creamos la tubería antes de crear el proceso hijo (antes de llamar a la función *fork()*), el proceso hijo realizará una copia de todo el contenido del proceso padre, incluyendo también una copia de la tubería. Por lo tanto, el proceso padre como el proceso hijo tendrán dos descriptores de la tubería abiertos y utilizables. En caso de haber ocurrido un error en la invocación de la función *pipe()*, esta retornará el valor de -1;

```
int descriptorTuberia[2];
int pid;

pipe(descriptorTuberia);

pid = fork();
```

No es recomendable que ambos procesos intenten escribir en la misma tubería, al menos salvo que posean mecanismos de sincronización. De ocurrir esta situación, los datos podrían mezclarse y resultarían en datos erróneos.

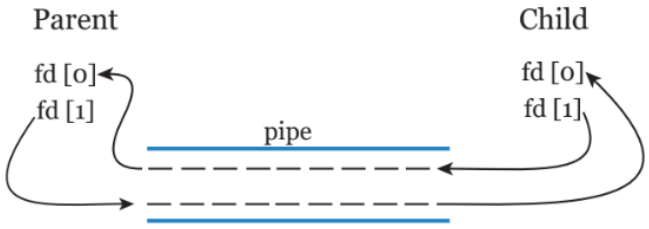


Figure 1: Creación de una Tubería(pipe)

En el caso de la lectura, si el proceso que ha escrito un dato intenta leer la tubería antes que el proceso hijo, recogerá su propio mensaje. Si ambos intentan leer a la vez, recogerán cada uno trozos del mensaje. Por ello, es mejor utilizar la tubería para que un proceso solo envíe datos y sea el otro sea el único encargado de recibirlos. Si queremos una comunicación bidireccional, es mejor abrir dos tuberías (llamar dos veces a la función *pipe()*).

Para evitar equivocaciones, y sobre todo para no mantener recursos ocupados innecesariamente, el proceso padre debe cerrar el lado de la tubería que no vaya a utilizar. El proceso hijo tiene su propia copia de ese lado, así que aunque el proceso padre lo cierre, para él sigue abierto y utilizable. El proceso hijo debe de cerrar el otro extremo.

En el siguiente ejemplo cerraremos el descriptor de lectura en el proceso padre y el de escritura en el proceso hijo. Una vez hecho esto, el padre pude escribir por *descriptorTuberia[1]* y el hijo leer por *descriptorEscritura[0]*, como si se tratara de un fichero normal. Obviamente ambos procesos deben estar de acuerdo en qué estructuras de datos se van a enviar.

```
//Proceso Padre
close(descriptorTuberia[0]);
...
write(descriptorTuberia[1], buffer, length);
...
```

```
//Proceso Hijo
close(descriptorTuberia[1]);
...
read(descriptorTuberia[0], buffer, length);
...
```

En el caso del proceso hijo, con la función *read()*, lo dejara bloqueado hasta que el proceso padre envié su dato. Ademas, En caso se este listo a leer los datos con la función *read()*, debemos de asegurar mediante un bucle de lectura completa de todos los datos enviados, ya que podemos obtener una cantidad parcial de la información enviada.

5 Actividad

1. Analice y describa la actividad que realiza el siguiente código. Explique como sucede el proceso de envió de información del proceso padre al proceso hijo.

```c
#include <sys/types.h>
#include <wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
  pid_t idProceso;
  int estadoHijo;
  int descriptorTuberia[2];
  char buffer[100];

  if (pipe (descriptorTuberia) == -1) {
    perror ("No se puede crear Tuberia");
    exit(-1);
  }
  idProceso = fork();

  if (idProceso == -1) {
    perror ("No se puede crear proceso");
    exit(-1);
  }

  if (idProceso == 0) {
    close (descriptorTuberia[1]);
    read (descriptorTuberia[0], buffer, 8);
    printf ("Hijo : Recibido \"%s\"\n", buffer);
    exit(0);
  }

  if (idProceso > 0) {
    close (descriptorTuberia[0]);
    printf ("Padre : Envio \"Ciencia\"\n");
    strcpy (buffer, "Ciencia");
    write (descriptorTuberia[1], buffer, strlen(buffer)+1);

    wait(&estadoHijo);
    exit(0);
  }
  return(1);
}
```

2. Analice y describa la actividad que realiza el siguiente código. Explique como sucede el proceso de envió de información del proceso padre al proceso hijo.

```c
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>

#define LEER 0
#define ESCRIBIR 1

char *frase = "Envia esto a traves de un tubo o pipe";
extern int errno;

int main(){
  int fd[2], bytesLeidos;
  char mensaje[100];
  int control;

  // se crea la tuberia
  control = pipe(fd);
  if ( control != 0 ) {
    perror("pipe:");
    exit(errno);
  }

  // se crea el proceso hijo
  control = fork() ;
  switch(control) {
    case -1 :
      perror("fork:");
      exit(errno);
    case 0 :
      close(fd[LEER]);
      write(fd[ESCRIBIR], frase, strlen(frase) + 1);
      close(fd[ESCRIBIR]);
      exit(0);
    default :
      close(fd[ESCRIBIR]);
      bytesLeidos = read(fd[LEER], mensaje, 100);
      printf("Leidos %d bytes : %s\n", bytesLeidos, mensaje);
      close(fd[LEER]);
  }
  exit(0);
}
```

3. Elabore un programa propio que emplee la comunicación entre procesos (padre e hijo) utilizando pipes.
4. Investigue como se pueden enviar datos de un proceso padre a un proceso hijo y viceversa.

6 Entregables

Al finalizar el estudiante deberá:

- Elaborar documento de sus actividades realizadas, escriba de forma clara el análisis realizado de cada punto planteado en la sección actividad. Incluya capturas de pantalla que demuestre su correcta ejecución.
- En cuanto al código desarrollado, deberá de presentar un video explicativo de sus soluciones y la ejecución del codigo propuesto, así como adjuntar el código fuente. Duración del video: 5 a 12 minutos
- En el último punto de la actividad, incluya un ejemplo breve de la doble comunicación.
- Deberán de subir a la plataforma el link al portafolio con el video, informe y codigo funte. Sólo es necesario una entrega por grupo

Rúbrica	Excelente	Bueno	Regular	Insuficiente
Informe y video: Desarrolla un informe, con un formato limpio y fácil de leer.	3	2	1	0
Ejercicios: Resuelve correctamente cada ejercicio propuesto	11	9	6	2
Presentación y dominio del tema: El alumno demuestra dominio del tema y conoce con exactitud cada parte del documento presentado	6	4.5	3	1

7 Rúbrica de Evaluación

- **IMPORTANTE** En caso de copia o plagio o similares todos los alumnos implicados tendrán sanción en toda la evaluación del curso.